University of the West of England

System Development Group Project - 2025-26

PARAGON APARTMENT MANAGEMENT SYSTEM

Group: A23

Team Member:

Lahiru Malshan Aluthgama Hewage	24058606
Duc Trung Nguyen	25036440
Tran Gia Ho	25044389
Loku Wellage Sadila Sathnidu Chandrakumara	24055479
Wijesinghe Arachchige Sihath Sanvidu Perera	24061015

Table of Contents

I. Introduction 4

II. Project Overview & Objectives 4

III. Literature Review 6

1. **Introduction** 6
2. **The Evolution of Property Management Systems (PMS)** 6
3. **Digital Transformation and Web-Based Architectures** 7
4. **Automation and Workflow Integration in PMS** 8
5. **Security and Role-Based Access Control (RBAC)** 8
6. **Usability and User Experience (UX)** 9
7. **The Rise of Tenant Self-Service Portals** 9
8. **Challenges in Multi-City Property Management** 10
9. **Conclusion** 10

IV. Requirements 11

1. Functional requirements 11

1.1. User / Account Management 11

1.2. Tenant Management 11

1.3. Apartment Management 12

1.4. Payment & Billing 12

1.5. Maintenance Management 12

1.6. Reporting 12

2. Non-Functional Requirements 12

2.1. Performance 12

2.2. Security 13

2.3. Scalability 13

2.4. Reliability 13

2.5. Usability 13

2.6. Maintainability 13

2.7. Consistency 13

2.8. Compatibility 14
V. Project Planning and Team Roles 14
1. Planning Phase 14
2. Design Phase 16
3. Implementation Phase 16
4. Conclusion 17
VI. Implementation and Testing 17
1. Development Phases and Software Components 17
2. Handling Requirements 21
2.1 Fully Integrated Functional Modules (UI + Database) 24
2.2 Partially Integrated & UI-Completed Modules 25
2.3 Non-Functional Achievements 25
3. Test Results and Evaluation 26
VII. Evaluation & Lessons Learned 27
1. Introduction 27
2. What We Learned as a Team 27
3. Achievements 28
4. Incomplete project and Teamwork Issues 28
5. Planning Evaluation 28
6. Teamwork Challenges and Lessons Learned 29
7. Evaluation Skills and Future Development 29
8. Conclusion 30
VIII. Reference 30
IX. Appendices 32

I. Introduction

Paragon Apartment Management Company is currently facing significant challenges in its development process. Despite successfully expanding its operations across major UK cities like Manchester, London, Cardiff, and Bristol, the company's internal administrative processes have not kept pace with its growth. One of the major barriers to this development is the organization's reliance on a decentralized, paper-based system. Because the regional offices operate in silos without a shared network, data is often duplicated, inconsistent, and security risks, especially user's personal data and financial information.

This outdated infrastructure is actively holding the Paragon's business growth. Routine tasks, such as processing new tenant applications or tracking rent collections, are done manually. This makes the workflow painfully slow and highly prone to human error. Furthermore, senior management can't access real-time reports of how the business is performing because financial and occupancy data are scattered across different locations. Besides its operational inefficiencies, the current system also lacks strict security protocols, particularly regarding user permissions. Without clear role-based permissions, user's personal data and information are at risk of unauthorized access. These issues highlight the urgent need for a modern, secure, and unified management solution.

To resolve these challenges, our team proposed the development of the Paragon Apartment Management System (PAMS) in digital context. PAMS is designed to replace the fragmented, paper-based system with a unified application platform. By centralizing the database, PAMS ensures that all updates regarding tenants in any of their city are instantly displayed in its head office. Also, the system will enforce Role-Based Access Control (RBAC) to ensure that each role only has access to their respective system. By automating rent tracking, maintenance requests, and reporting, PAMS will not only solve Paragon's immediate administrative chaos but also provide a solid, scalable foundation for future expansion. This report will present the system requirements, key features, and expected benefits of implementing PAMS.

II. Project Overview & Objectives

Project Name: Paragon Apartment Management System (PAMS)

Client: Paragon Apartment Management

Problem: The client's current decentralized, paper-based operations across multiple UK locations (Bristol, Cardiff, London, Manchester) are inefficient, error-prone, and insecure. This leads to data inconsistency, poor scalability, and a lack of real-time business insights.

Proposed Solution: PAMS will be a centralized, secure, and scalable application software solution. It will automate key business processes, enforce consistent data handling, and provide robust, role-based access control (RBAC) for all stakeholders. The system will unify operations, enhance security, and deliver real-time reporting, enhancing data integrity, capped by a modern, self-service dashboard for tenants.

Project Aims:

The primary aim of this project is to design, develop, and implement a consolidated, scalable, and secure apartment management software solution for the Paragon company. This system will centralize operations, automate manual processes, provide, enhance data integrity, and support multi-city growth.

Core Objectives:

- **Centralize Data:** Consolidate all apartment, tenant, financial, and maintenance data into a single, secure database to eliminate duplication and inconsistency.
- **Implement Robust RBAC:** Secure the system by implementing a strict, multi-layered RBAC model for six distinct user roles (Administrator, Manager, Front-desk, Finance, Maintenance, Tenant).
- **Automate Workflows:** Digitize and automate critical processes, including tenant registration, rent billing, and maintenance request tracking.
- **Enhance Usability (Non-Function Requirements):** Deliver an intuitive, user-friendly interface for both staff and tenants to reduce training time and increase adoption.
- **Provide Real-Time Reporting:** Equip management with powerful reporting tools for instant insights into occupancy, financials, and maintenance performance.

- **Ensure Scalability (Non-Functional Requirement):** Build the system on a modern, scalable architecture that allows Paragon to easily add new cities and properties as the business expands.

III. Literature Review

1. Introduction

Traditional real estate management has usually relied on manual processes and separate administrative tasks. However, with the ongoing shift towards digital transformation, these methods are becoming less effective. Tasks such as tenant management, rent collection, maintenance tracking, and financial handling are increasingly difficult to manage using traditional approaches alone.

As property companies expand and operate across multiple locations, these older methods often struggle to keep up with operational demands. This can result in issues such as duplicated data, inconsistent records, and potential security risks. This section provides an overview of the project background, reviews relevant academic research, and introduces key technologies. It also highlights the main limitations of existing systems, which the Paragon Apartment Management System (PAMS) aims to address.

2. The Evolution of Property Management Systems (PMS)

Historically, property management relied heavily on manual and time-consuming processes. Many organisations used physical records or simple spreadsheet tools to manage tenancy agreements, rent payments, and maintenance requests. While this approach could work for small landlords, *Baum (2017)* [1] points out that it becomes difficult to sustain as the organisation grows. Manual data entry also increases the risk of human error, which can lead to duplicated records and financial inconsistencies. These issues are clearly reflected in the Paragon case study.

The move towards digital Property Management Systems (PMS) is mainly driven by the need to improve efficiency. *Ullah et al (2018)* [2] suggest that adopting smart technologies helps modern real estate companies overcome the limitations of older systems. For example,

tasks such as rent invoicing and lease renewal reminders can be automated, allowing staff to focus on more important activities like tenant support and business growth. In addition, *Saarikko et al. (2020)* [3] highlight that digital transformation is not only about using new technology, but also about changing how a business operates to ensure better coordination between processes. For Paragon, adopting a digital PMS is therefore not just a technical improvement, but a necessary step to address the delays and inefficiencies in handling tenant applications and rent collection.

3. Digital Transformation and Web-Based Architectures

Digital transformation in real estate focuses on using modern technologies to centralise data, standardise processes, and improve access across different parts of an organisation. It also aims to enhance the overall quality of service. A study published in the *International Journal of Information Management Data Insights* [4] shows how technologies such as AI, IoT, and digital twins are changing property management processes and creating more value for stakeholders, including managers, investors, tenants, and service providers. The study suggests that these technologies do not just improve existing systems, but also reshape how real estate operations are managed, particularly in areas such as decision-making and automation.

Although technologies like AI and IoT are becoming more important, a key development for organisations operating in multiple locations is the move from local systems to centralised web-based or cloud-based solutions. Research shows that web-based systems are especially useful for managing properties spread across different locations. *Marston et al. (2011)* [5] explain that, unlike traditional desktop software installed on individual computers, web-based systems allow real-time access to data and better synchronisation between locations. This makes them more scalable and easier to manage. It also helps solve the common problem of data silos, where information is stored separately in different branches.

For Paragon, adopting a web-based system means that data from cities such as Bristol, Cardiff, London, and Manchester can be stored and managed in one central system. This creates a single, consistent source of information and reduces the risk of errors caused by unsynchronised data in their current setup.

4. Automation and Workflow Integration in PMS

Automation plays an important role in improving property management operations. Research and industry reports show that automating routine tasks—such as billing notifications, lease renewal reminders, and maintenance tracking—can significantly reduce workload and improve accuracy. Compared to manual processes, automated systems can complete these tasks more quickly and with fewer errors. According to *MoldStud (2023)* [6], automation also helps integrate workflows by ensuring that tasks follow a consistent and predefined process, which reduces variation and mistakes. For instance, an automated maintenance system can record, assign, and track repair requests in a structured way. Similarly, automated billing systems help reduce missed payments and make financial management more predictable.

In addition to improving efficiency, automation also supports better oversight and control. Automated systems can generate real-time reports and maintain clear audit trails, allowing managers to monitor key performance indicators more effectively. *Deloitte (2024)* [7] notes that such reporting tools can improve financial forecasting by providing up-to-date information on occupancy rates and outstanding payments. In the case of Paragon, introducing workflow automation would help reduce delays and errors caused by manual handling of tenant applications and rent collection. However, existing research also points out that automation needs to be implemented carefully to ensure data accuracy and to avoid spreading errors across the system.

5. Security and Role-Based Access Control (RBAC)

As PMS platforms become more centralised and automated, ensuring data security is increasingly important. These systems store sensitive information such as tenants' personally identifiable information (PII), financial records, and contractual documents. Without proper access controls, there is a risk of unauthorized access, data breaches, and non-compliance with regulations.

Role-Based Access Control (RBAC) is widely regarded as an effective method for managing permissions in multi-user systems. RBAC assigns access rights based on predefined roles rather than individual users, so employees can only access information relevant to their responsibilities. *Sandhu et al. (1996)* [8] note that RBAC both improves system security and simplifies access management. In property management, common roles include

administrators (full access), managers (reporting access), and operational staff (operational access). This setup reduces insider threats and helps organisations comply with data protection regulations, such as the GDPR. *Voigt and Von dem Bussche (2017)* [9] emphasize that clearly defined access roles are essential for meeting modern European data privacy standards. For Paragon, implementing RBAC is crucial to address specific access and security issues identified in the case study, ensuring that maintenance staff cannot view financial data and vice versa.

6. Usability and User Experience (UX)

While technical architecture and security form the foundation of a Property Management System, its success also depends heavily on usability and user experience (UX). For Paragon, where staff are used to manual processes and physical records, moving to a digital platform is a significant change. If the system interface is complex or not intuitive, staff may resist using it, training costs may rise, and human errors—those the system is meant to reduce—may continue to occur.

Good UX design helps prevent data inconsistencies. Standardised input forms, mandatory fields, and real-time validation within the interface can stop staff from entering incomplete tenant information or incorrect financial data. Data visualisation is also essential for decision-making. Instead of showing managers raw database entries, a modern PAMS should provide dashboards that turn complex data into clear graphs and charts. This visual approach allows Paragon's management to quickly see occupancy trends and payment statuses across cities such as Bristol, Cardiff, London, and Manchester, supporting faster and more accurate decisions without needing deep technical knowledge.

7. The Rise of Tenant Self-Service Portals

Modern property management is increasingly moving towards a tenant-focused approach. Research suggests that Self-Service Technologies (SST) can improve tenant satisfaction and increase operational efficiency. *Blut et al. (2016)* [10] found that users feel more control and convenience when they can complete tasks independently through digital portals. By offering a dedicated "Tenant Dashboard," organisations allow residents to manage tasks such as

checking payment history, validating card payments, or logging complaints without needing staff assistance.

Studies also show that self-service portals help reduce administrative workload and improve data accuracy, since information is entered directly by tenants. This fits Paragon's needs perfectly, as the system should enable tenants to "view progress reports of repair requests" and "see graphical representations of payment history," promoting transparency and reducing unnecessary communication.

8. Challenges in Multi-City Property Management

Managing properties across multiple cities adds more complexity compared to single-location organisations. Research highlights challenges such as fragmented data, inconsistent operational procedures, communication delays, and difficulties in producing consolidated reports. When regional offices work independently, differences in data formats and update schedules can result in unreliable or incomplete information.

Many studies also point out that existing PMS solutions are often not fully adapted to multi-city operations. Some systems lack real-time synchronization, while others offer limited support for centralised oversight. This makes it hard for senior management to obtain a complete picture of organisational performance. *IBM (2020)* [11] suggests that enterprises operating across multiple locations need systems that balance central control with local flexibility.

For Paragon, the challenges of multi-city management are clear. Data from offices in Manchester, London, Cardiff, and Bristol are not fully synchronised. Therefore, a PMS designed for multi-location operations must provide standardised data formats, real-time reporting, and scalable architecture to support future growth.

9. Conclusion

Existing literature on property management systems and digital transformation shows a clear pattern: organisations that successfully implement digital tools experience improved efficiency, better data quality, and stronger operational control. Studies emphasise the importance of web-based architectures for multi-site coordination, RBAC for detailed security management, and self-service portals to enhance user experience. Despite these

advances, challenges remain in areas such as multi-city coordination, comprehensive security, and full workflow coverage.

The development of the Paragon Apartment Management System (PAMS) is therefore grounded in these established theoretical frameworks. By combining centralised data management, automation, robust RBAC security, and tenant-focused technologies, the system aims to overcome the limitations of existing solutions. This approach supports scalable, secure, and efficient property management, tailored to the needs of organisations like Paragon.

IV. Requirements

1. Functional requirements

1.1. User / Account Management

- The system must allow Front-desk staff, Finance managers, Maintenance staff, Administrators, Managers, and Tenants to log in.
- The system must enforce role-based access control (different users see different features).
- The system must let Admins create, update, and delete staff's accounts, also approve tenant account.
- Front-desk staff must be able to register new tenants with the required info (NI number, name, phone, email, occupation, references, apartment requirement, and lease details).
- Admins must manage apartments for their assigned location.

1.2. Tenant Management

- The system must allow staff to add, update, and delete tenant records.
- The system must track payment history, notifications, maintenance logs, and lease agreements.
- The system must allow tenants to request early contract termination and calculate the 5% penalty.

1.3. Apartment Management

- The system must register apartments with attributes (location, type, number of rooms, rent).
- The system must assign apartments to tenants and track occupancy status.
- The system must track maintenance requests linked to each property.

1.4. Payment & Billing

- The system must generate invoices and record payments.
- The system must identify and notify about late payments.
- The system must generate receipts and store payment history.
- The system must calculate pending vs collected rent.

1.5. Maintenance Management

- Tenants must be able to submit maintenance requests.
- Maintenance staff must investigate issues, prioritise them, and update status.
- The system must log resolution details, time taken, and cost.
- The system must store communication dates and times with tenants.

1.6. Reporting

- The system must generate occupancy reports (by apartment or city).
- The system must produce financial summaries (collected vs pending rent).
- The system must track maintenance costs.
- The system must allow managers to view business performance across all cities.

2. Non-Functional Requirements

2.1. Performance

- The system must handle multiple cities and users without slowing down.

- Reports should be generated within a reasonable time (e.g., <3 seconds for large datasets).
- Dashboard graphs should load smoothly without lag.

2.2. Security

- The system must enforce role-based access control (RBAC).
- Sensitive data (NI number, payments) must be stored securely.
- The system must prevent unauthorized access between cities.

2.3. Scalability

- The system must support expansion to new cities without rewriting the codebase.
- The database must handle growing tenants, apartments, and maintenance records.

2.4. Reliability

- The system must be available with minimal downtime.
- Data must not be lost during crashes (use backups/transactions).

2.5. Usability

- The UI must be intuitive for staff and tenants.
- Staff should be able to use the system with minimal training.
- The system should have a clean dashboard for tenants.

2.6. Maintainability

- Code should be modular and easy to update.
- New features should be addable without breaking existing ones.

2.7. Consistency

- Tenant data must stay synchronized across all cities.
- Duplicate records must be prevented.

2.8. Compatibility

- The system should run smoothly on Windows, Mac, and Linux (if web-based: it runs on all browsers).
- Mobile-friendly UI for tenants (optional but ideal).

V. Project Planning and Team Roles

Member	Roles
Duc Trung Nguyen	Project Manager, Requirement Analyst, Database Designer, System Analyst
Lahiru Malshan Aluthgama Hewage	Implementation Lead, Backend Developer, System Analyst
Wijesinghe Arachchige Sihath Sanvidu Perera	Database Designer , Support Development, Debugging
Tran Gia Ho	Frontend Development, UI Design, System Design, QA Tester
Loku Wellage Sadila Sathnidu Chandrakumara	QA Testing, Backend Developer, Debugging, System Design

Figure 1 Team Roles Table

This project followed a structured development process consisting of three main phases: planning, design, and implementation. Each phase contributed to building a functional and well-organized apartment rental management system.

1. Planning Phase

The planning phase focused on defining the system idea, objectives, and requirements. The team first discussed and selected the concept of an apartment rental management system to address real-world problems such as tenant management, maintenance handling, and role-based access.

We then defined the main objectives of the system, which include providing a centralized platform for different users, improving data management, and supporting efficient communication between stakeholders. To manage the project timeline, we created a Gantt chart that helped allocate tasks, set deadlines, and track progress throughout the project.

Additionally, we identified both functional and non-functional requirements. Functional requirements included user registration, login, role-based dashboards, tenant management, maintenance requests, and payment processing. Non-functional requirements focused on system security, performance, scalability, and usability, ensuring the system is reliable and user-friendly.

Phase	Task	Description	Starting Date	Due Date	Roles Responsible
Planning	Idea Generation	Identify and select project idea	13/10/2025	1 week	Project Manager, System Analyst
	Define Objectives	Set system goals and scope	20/10/2025	1 week	Project Manager
	Requirement Analysis	Define functional & non-functional requirements	27/10/2025	1 week	System Analyst, System Designer
	Gantt Chart Creation	Plan timeline and task allocation	03/11/2025	2 weeks	Project Manager
Design	Class Diagram	Define system classes and relationships	17/11/2025	1 week	System Designer
	Use Case Diagram	Model system interactions	17/11/2025	1 week	System Designer
	ERD Design	Design database structure	01/11/2025	1 week	System Designer, Database Designer
	System Architecture	Design overall system structure	15/12/2025	1 week	System Architect, Backend Developer
Implement	Database Development	Create tables and relationships	26/01/2026	2 weeks	Backend Developer
	Frontend Development	Build UI with Python + Flet	26/01/2026	2 weeks	Frontend Developer
	Backend Development	Implement login, register, role logic	09/02/2026	3 weeks	Backend Developer
	Dashboard Development	Develop role-based dashboards	23/02/2026	2 weeks	Frontend Developer, Backend Developer

Phase	Task	Description	Starting Date	Due Date	Roles Responsible
Testing	Functional Testing	Test system features	09/03/2026	2 weeks	QA Tester, Frontend Developer
	Bug Fixing	Identify and fix errors	16/03/2026	2 weeks	Backend Developer, Frontend Developer, Support Development
	Final Validation	Ensure system meets requirements	23/03/2026	1 week	QA Tester, Project Manager

Figure 2 Gantt Chart Table

2. Design Phase

In the design phase, we transformed the requirements into system models and diagrams. An Entity-Relationship Diagram (ERD) was created to represent the database structure and relationships between entities such as users, roles, tenants, apartments, and maintenance requests. This ensured proper data organization and consistency.

We also developed class diagrams to define the structure of the application, including key classes and their relationships. In addition, sequence diagrams were used to illustrate system processes such as login, registration, and maintenance request handling. These diagrams provided a clear understanding of system interactions and data flow, which supported accurate implementation.

3. Implementation Phase

In the implementation phase, we developed a desktop application using Python and the Flet framework. The system features a clear and user-friendly interface, making it accessible for different types of users.

The application supports multiple roles, including administrator, manager, maintenance staff, front-desk staff, and tenants. Each role has a separate dashboard to ensure security and proper access control.

The administrator can manage tenants and system data. The manager is responsible for analyzing rental data and approving tenant accounts. Front-desk staff can create tenant accounts, which require approval from the manager. Maintenance staff can handle maintenance requests submitted by tenants. Tenants can make payments, submit maintenance requests, and receive notifications.

This role-based system ensures that users can only access functions relevant to their responsibilities, improving both system security and usability.

4. Conclusion

Overall, the project followed a clear and structured process from planning to implementation. Each phase contributed to building a complete and functional system. This approach improved organization, reduced development errors, and ensured the system meets user requirements effectively.

VI. Implementation and Testing

This section details the practical construction of the Paragon Apartment Management System (PAMS), outlining the development lifecycle, the technologies used, and an objective evaluation of the testing phase.

1. Development Phases and Software Components

The development of PAMS was carried out using a a modern technology suite, focusing on Python. We chose Python in combination with Flet for the frontend user interface. Flet was chosen because it allows for the rapid development of interactive, cross-platform applications without needing to write separate HTML/CSS/JavaScript. For the backend data management, we used MySQL, which provides a robust relational database for handle complex entities like tenants, apartments, maintenance, and financial records.

The project was divided into three distinct implementation phases:

Phase 1: Conceptualization and Design: Before writing any code, our team focused on mapping out the system logic. We created UML Class Diagrams and Use Case Diagrams to

visualize how different actors (Tenants, Administrators, Staffs) would interact with the database.

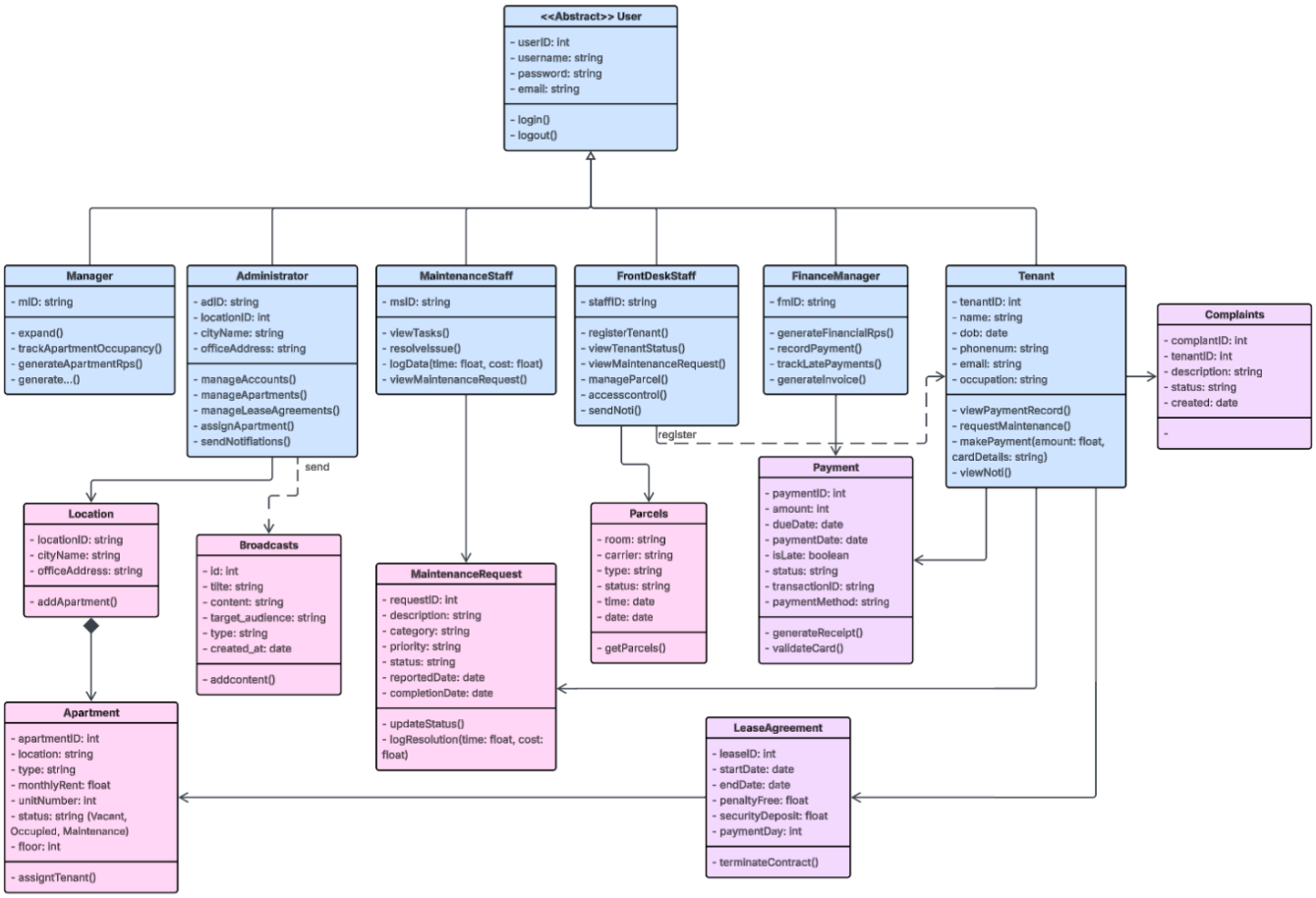

Figure 3 UML Diagram

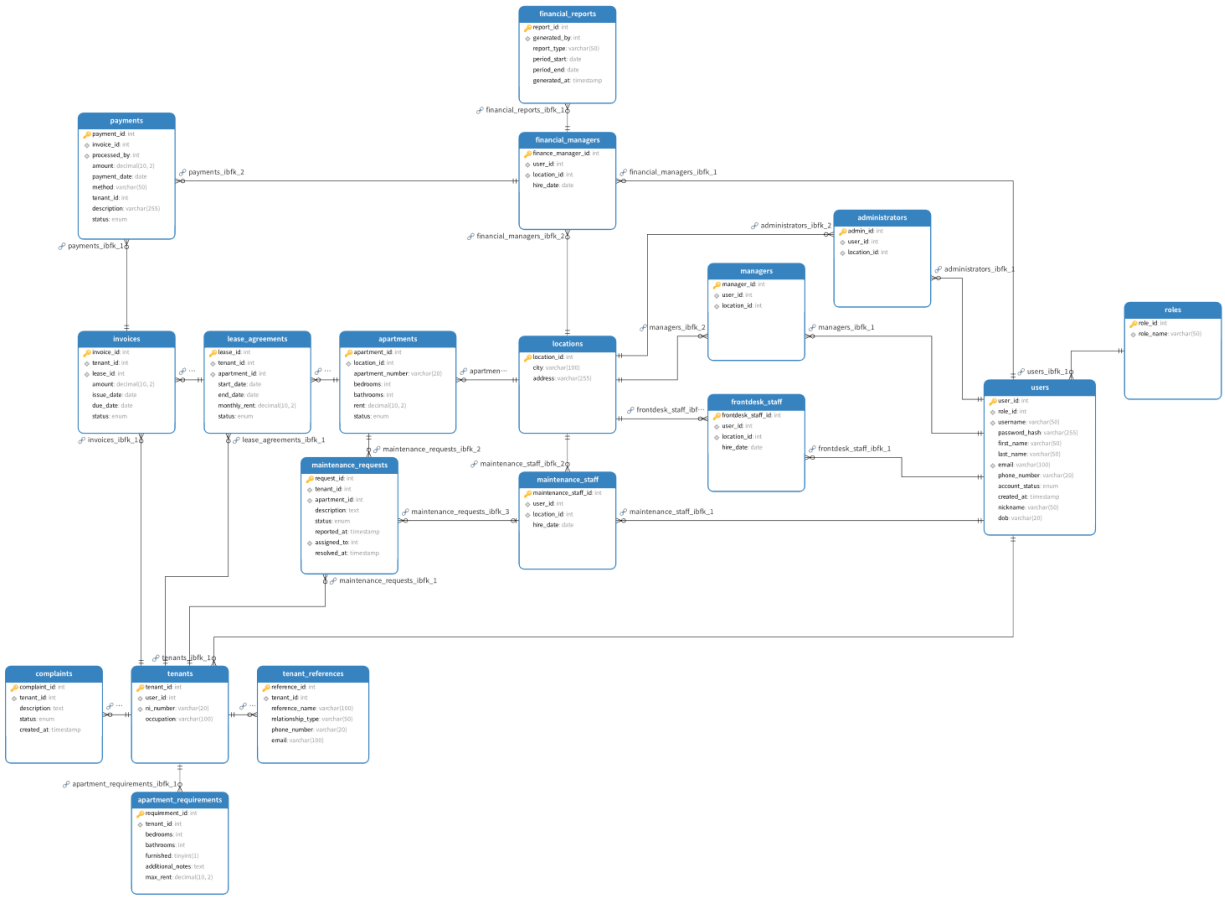

Figure 4 ERD Diagram

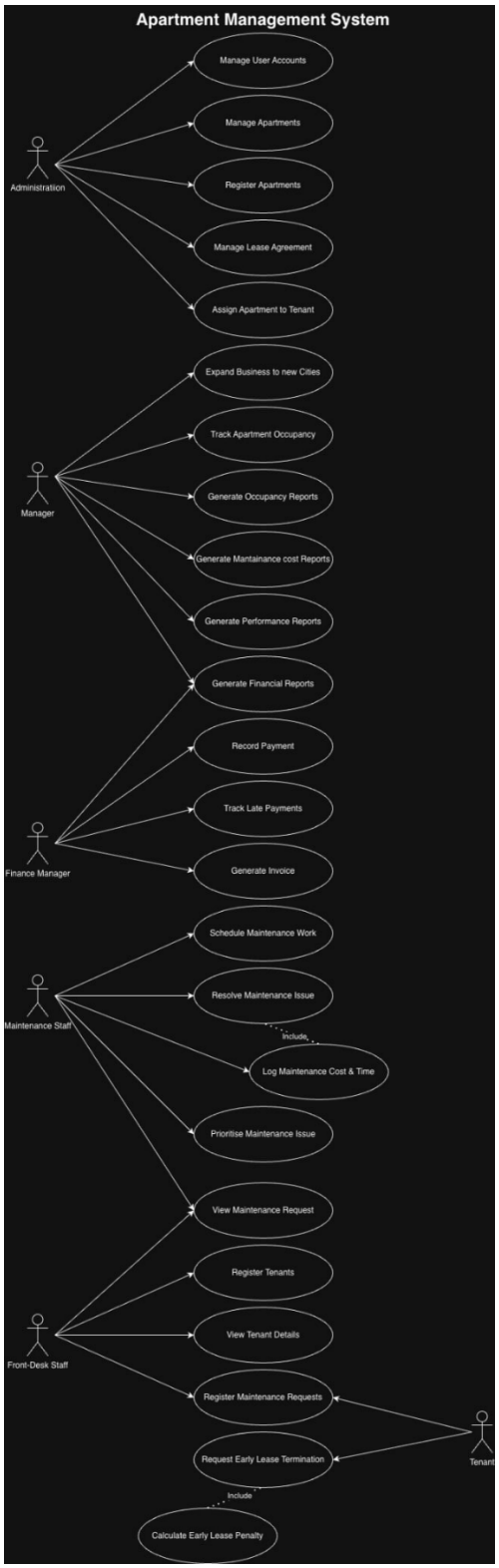

Figure 5 Use Case Diagram for PAMS

Phase 2: Parallel UI and Database Construction: To maximize efficiency, our team split into two sub-groups. One group focused on building the MySQL database schema (tables, foreign keys, and constraints). The other group focused on building the graphical user interface using Flet.

Phase 3: Integration and Backend Logic: The final phase involved connecting the Flet frontend to the MySQL database. We wrote SQL queries within our Python backend to ensure that actions performed on the UI (e.g., submitting a maintenance request) would directly update the database records.

2. Handling Requirements

Reflecting on the Requirements outlined in Section IV, the team made significant progress, successfully delivering the core interactive components of the system.

Currently, the Graphical User Interface (UI) is approximately **95% complete** including Functional and Non-Functional Requirements for all user roles. The system features distinct, visually cohesive dashboards for every role type.

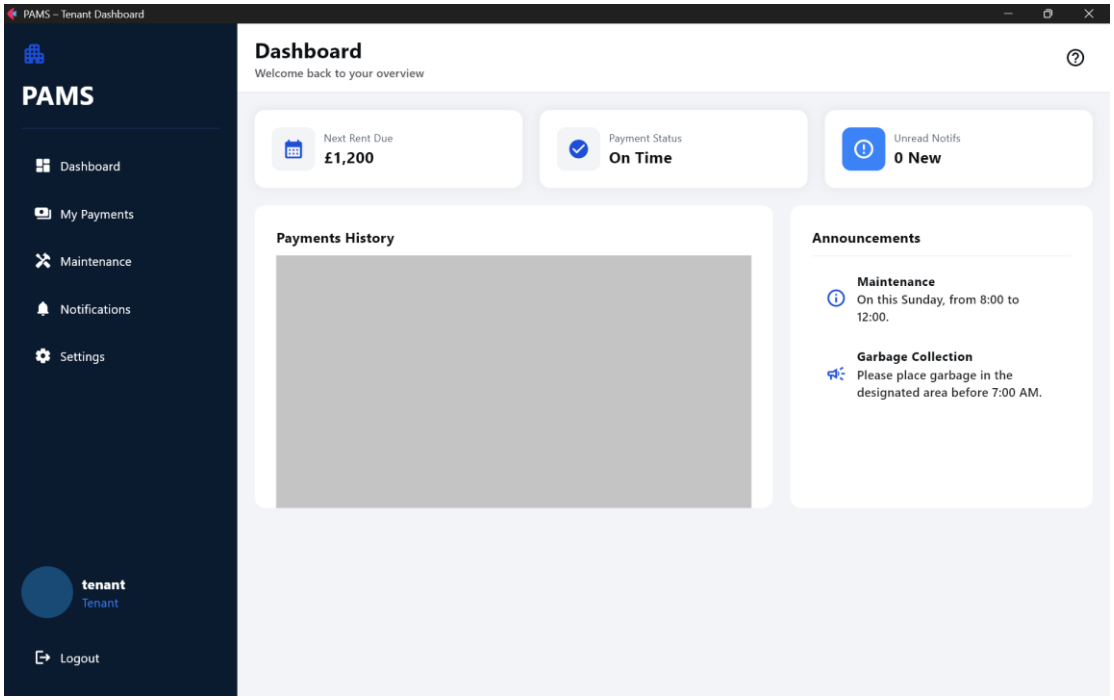

Figure 6 Tenant's Dashboard

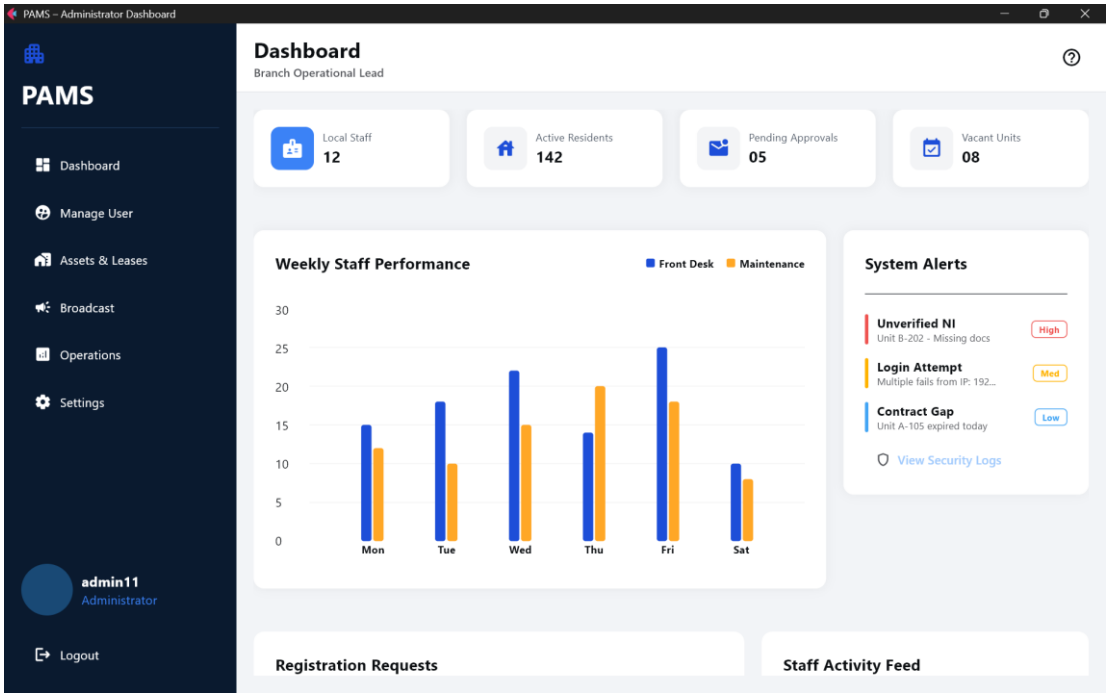

Figure 7 Administrator's Dashboard

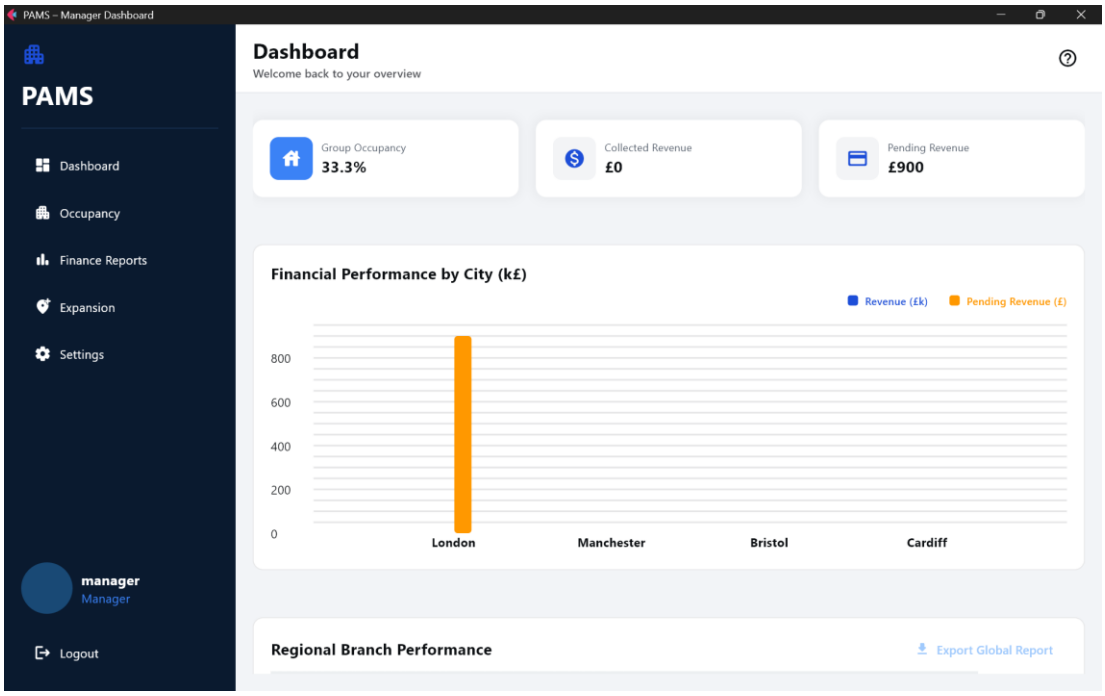

Figure 8 Manager's Dashboard

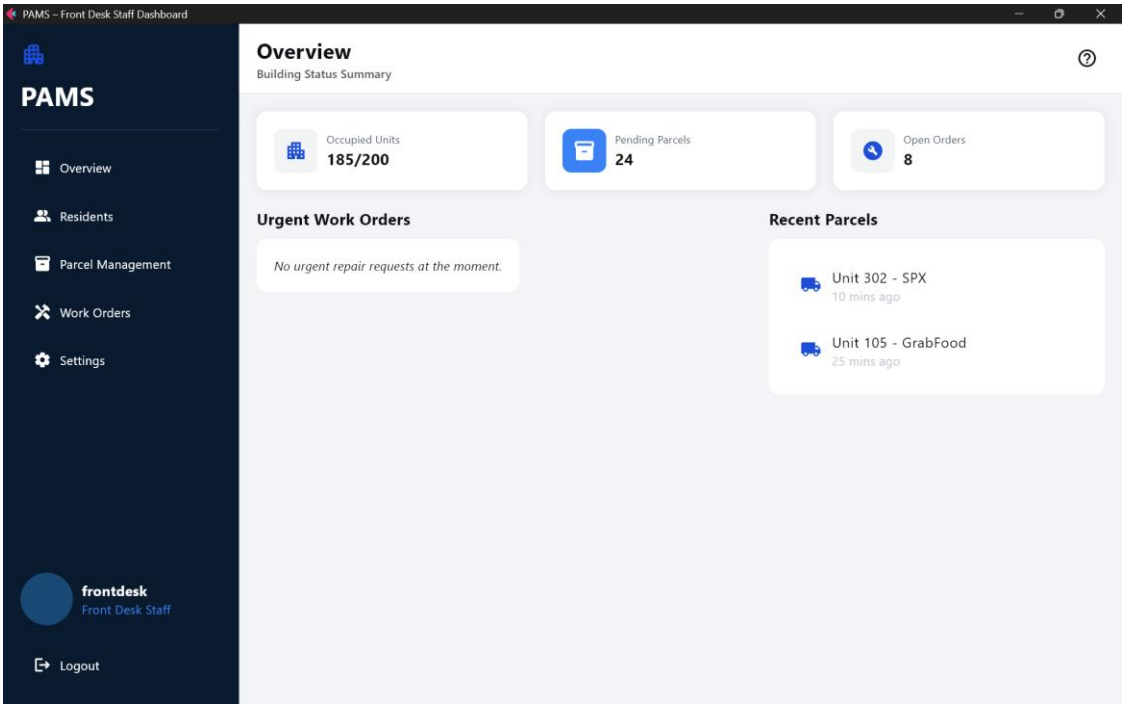

Figure 9 Front Desk Staff's Dashboard

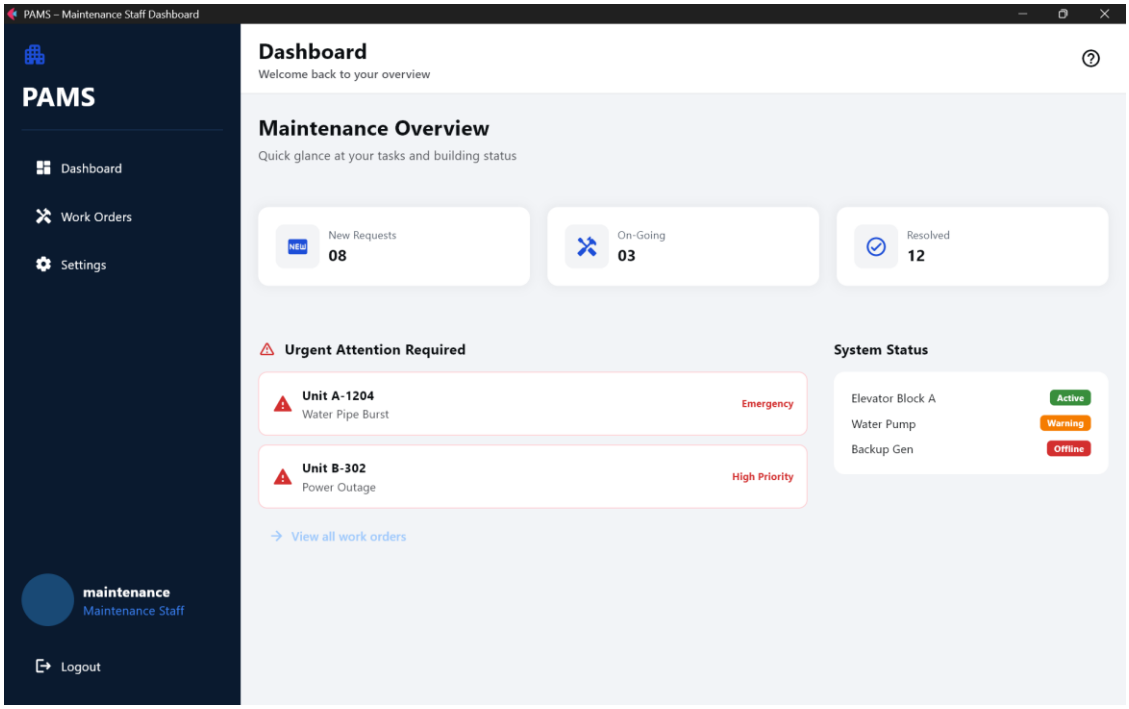

Figure 10 Maintenance Staff's Dashboard

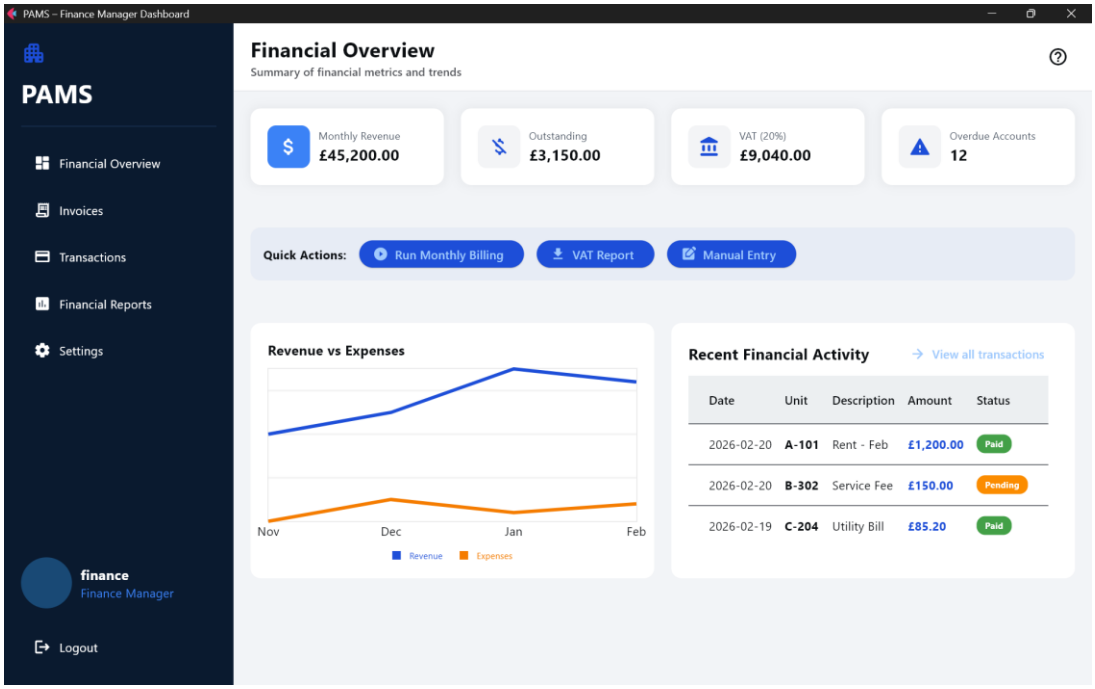

Figure 11 Finance Manager's Dashboard

In terms of functional and non-functional, database-connected features, we successfully completed the following critical requirements:

2.1 Fully Integrated Functional Modules (UI + Database)

- **Tenant Management Module:** Tenants can successfully view their profiles, manage system settings, submit Maintenance Requests, process Payments, and receive real-time notifications. All these actions are correctly recorded and updated in the live database.
- **Administrator Module:** Fully functional. Administrators have higher privileges to send notifications, manage staff and tenant accounts (CRUD operations), view the overall maintenance dashboard, and monitor company assets and lease agreements.
- **Maintenance Staff Module:** Integrated to streamline repair workflows. Maintenance staff can view their specific work order boards, send internal maintenace requests, accept assigned maintenance jobs, and submit digital completion forms once a task is finished, instantly updating the database status.
- **Authentication & Security:** The Login and Registration systems are fully operational. To ensure data protection, user passwords are securely **hashed** before being stored in the

MySQL database. The system securely verifies credentials against the MySQL database and successfully routes users to their specific dashboards based on their Role-Based Access Control (RBAC) level.

2.2 Partially Integrated & UI-Completed Modules

- **Front Desk Staff Module:** The critical function of registering and inputting new resident forms is successfully connected to the database. Remaining features (e.g., parcel management, approving maintenance requests) possess a fully smooth and responsive UI but are currently pending backend integration.
- **Manager & Finance Manager Modules:** The graphical user interfaces for both the Manager and Finance Manager roles are fully developed, visually cohesive, and run smoothly. However, the complex backend SQL queries required for these analytical and financial dashboards are currently utilizing mock data and await final database connection.

2.3 Non-Functional Achievements

- **Cross-Platform Compatibility:** Leveraging the Flet framework, the PAMS application is highly compatible and runs natively across Windows, macOS, and Linux operating systems without requiring separate codebases.
- **Data Consistency & Integrity:** The centralized MySQL database ensures that data is synchronized across all city branches. By implementing Primary Keys and unique User IDs, the system strictly prevents duplicate records, ensuring a single source of truth.
- **Usability:** Despite the complex backend logic, the frontend was designed with a strong focus on Human-Computer Interaction (HCI). The UI is highly intuitive and user-friendly, allowing non-technical staff to navigate their daily workflows efficiently.

While the UI is built for the remaining administrative and financial roles, some specific underlying backend logic is still under development, meaning certain buttons currently act as visual placeholders rather than executing database queries

3. Test Results and Evaluation

Throughout the development of the Paragon Apartment Management System (PAMS), testing was carried out continuously to ensure both the responsiveness of the Flet-based user interface and the integrity of the MySQL database backend. The testing approach mainly focused on manual functional testing and integration testing, where each user flow was checked against the original system requirements.

Here are some pictures of our core test cases:

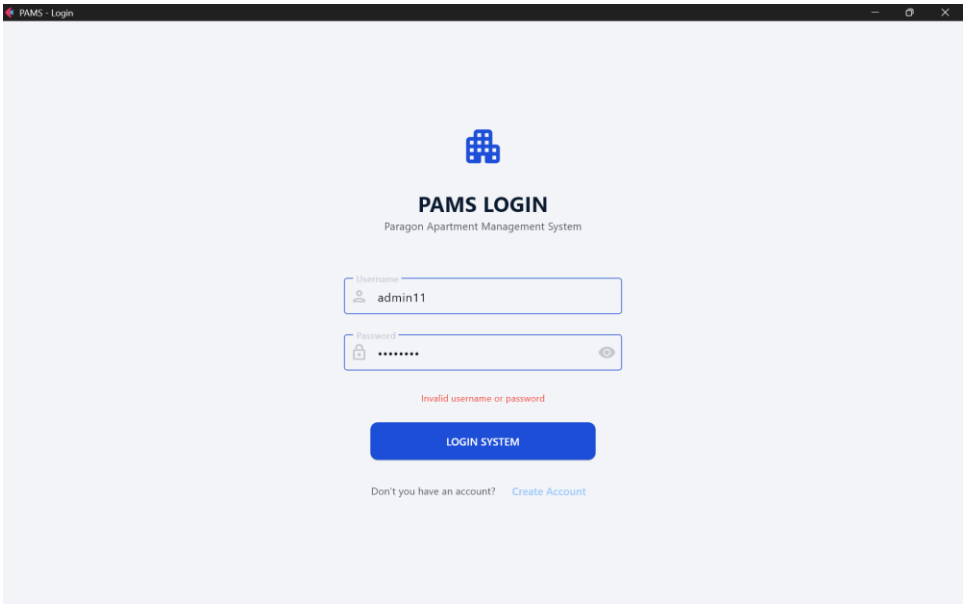

Figure 12 Reponse when incorrect input occurs

During testing the authentication process and Role-Based Access Control (RBAC), t these security features worked as expected. After logging in, users were correctly redirected to their respective dashboards (such as Tenant, Administrator, Maintenance, Manager, Front Desk Staff, or Finance Manager), while access to restricted areas was properly blocked.

For the fully integrated modules, such as the Tenant and Administrator portals, end-to-end testing produced positive results. Core Create, Read, Update, and Delete (CRUD) operations—such as updating tenant information, submitting maintenance requests, and managing staff accounts— worked reliably without issues like data duplication or database conflicts. The use of unique User IDs also helped maintain consistency across simulated multi-city scenarios.

When evaluating the system as a whole, most core features passed the integration tests successfully. However, the testing phase also remained some limitations. Modules such as the Finance Manager and Manager dashboards were tested mainly at the UI level and performed consistently across different operating systems, including Windows, macOS, and Linux. Since these modules are currently using mock data, full backend testing has not yet been completed. Overall, the results suggest that the system architecture is stable and secure, and it provides a solid foundation for future development and full database integration.

VII. Evaluation & Lessons Learned

1. Introduction

This section provides a collective account of our team's experience developing the Paragon Apartment Management System (PAMS). Building a full-stack application from scratch was a significant step up from our previous individual coding assignments. It required not only technical skills, but also to coordinate tasks, align ideas, and integrate different components into one system. While we managed to deliver a functional user interface and core database features, the process reveals several weaknesses in how we worked as a team. Therefore, this section reflects on both our achievements and limitations, with a particular focus on how we worked as a team and what we learned from that experience.

2. What We Learned as a Team

One of the most important lessons we learned is that software development is not just about coding. In practice, a large part of the work involves communication, planning, and coordination between team members. At the beginning, we underestimated this aspect and focused mainly on individual tasks. However, as the project progressed, it became clear that even well-written code could not function properly without proper integration.

Another key lesson was the importance of clear communication. In the early stages, our team faced several misunderstandings due to language differences and unclear explanations of system requirements. This led to duplicated work and inconsistent implementation across different modules. Over time, we realised that regular meetings and open discussions were necessary to align our understanding. By sharing ideas more actively, we were able to reduce confusion and improve collaboration.

We also learned that teamwork requires flexibility and compromise. Not all ideas could be implemented, and sometimes we had to prioritise practical solutions over ideal ones. This experience helped us understand how to evaluate different approaches and make decisions as a group rather than individually.

3. Achievements

Despite the challenges, the team performed well during the initial design phase. We invested time in creating UML diagrams and planning the database structure before starting implementation. This helped us build a shared understanding of the system and reduced major design changes later.

From a teamwork perspective, this phase showed our ability to collaborate effectively when the goals were clearly defined. Each member contributed ideas, and decisions were made collectively. This strong start gave us a stable foundation for the technical development that followed.

4. Incomplete project and Teamwork Issues

Although the system appears complete from a UI perspective, several features are not fully functional. This reflects not only technical limitations but also issues in teamwork and coordination.

One major issue was uneven contribution across the team. While all members were involved, the level of contribution varied at different stages of the project. This created pressure on certain members and slowed down overall progress. In some cases, tasks were delayed because responsibilities were not clearly assigned or followed up.

Another issue was the lack of continuous integration. Different parts of the system were developed separately without frequent testing together. As a result, when we attempted to combine components, conflicts and errors appeared. This shows that our teamwork lacked a structured workflow for integration.

5. Planning Evaluation

In hindsight, our initial planning was only partially effective. While the early design phase was strong, our overall development strategy created problems later.

Separating frontend and backend into two large phases meant that integration was delayed until the final stage. This made it difficult to identify issues early and led to incomplete functionality in some modules. From a teamwork perspective, this approach reduced collaboration between frontend and backend developers, as each group worked in isolation for too long.

This experience showed us that planning should not only define tasks but also consider how different parts of the system will interact throughout development.

6. Teamwork Challenges and Lessons Learned

A major challenge throughout the project was communication inconsistency. In the early stages, tasks were distributed based on general roles, but without clear tracking or follow-up. As a result, some tasks were delayed, while others were completed without proper coordination. This created an imbalance in workload and affected the overall progress of the project.

We also noticed that when responsibilities were not clearly defined, team members sometimes assumed that someone else would complete a task. This led to gaps in the system where certain features were left unfinished.

Integration was also difficult. Since components were developed separately, combining them revealed issues such as mismatched data and missing features. This emphasised the importance of continuous integration and regular testing.

Over time, our teamwork improved. We moved from working individually to collaborating more actively, supporting each other and sharing ideas. Although this change came later in the project, it helped us complete the system.

7. Evaluation Skills and Future Development

From a teamwork perspective, we recognise that our current skills are still at an early stage. While we were able to collaborate and complete the project to a certain extent, we lacked structured methods for managing teamwork in a software project.

One key area for improvement is version control. We mainly used Git for basic file sharing, but we did not fully apply branching strategies or proper collaboration workflows. This increased the risk of conflicts and made integration more difficult.

Another important area is project management. In future projects, we need to adopt a more iterative approach, such as Agile, where tasks are broken into smaller parts and tested continuously. This would improve both teamwork and system quality.

We also need to improve communication habits. Regular updates, clearer task assignments, and more active discussions can help prevent misunderstandings and improve overall efficiency.

8. Conclusion

Overall, this project was both challenging and valuable. Although we did not complete every feature, the experience helped us understand the importance of teamwork in software development. The issues we faced - such as communication gaps, uneven contribution, and poor integration—highlight areas we need to improve. At the same time, these challenges provided practical lessons that cannot be learned from theory alone. As a team, we leave this project with a clearer understanding of how real-world software development works, and with a stronger awareness of the importance of collaboration.

VIII. Reference

List of references

[1] A. Baum, "PropTech 3.0: The Future of Real Estate," Saïd Business School, University of Oxford, 2017.

[2] Ullah, F.; Sepasgozar, S.M.; Wang, C., "A Systematic Review of Smart Real Estate Technology: Drivers of Adoption and Barriers," *Sustainability,* vol. 10, no. 9, 2018.

[3] Saarikko, T.; Westergren, U.H.; and Blomquist, T, "Digital transformation: Five recommendations for the digitally conscious firm," *Business Horizons,* 2020.

[4] Basheer Al-haimi; Haliyana Khalid; Nor Hidayati Zakaria; Tuti Haryati Jasimin, "International Journal of Information Management Data Insights," *Digital transformation in real estate and its challenges,* 2025.

[5] Marston, Sean; Li, Zhi; Bandyopadhyay, Subhajyoti; Zhang, Juheng; Ghalsasi, Anand, "Cloud computing — The business perspective," *Decision Support Systems,* vol. 51, no. 1, 2011.

[6] MoldStud, "The role of automation in enhancing property management systems," MoldStud, 2023. [Online]. Available: https://moldstud.com/articles/p-the-role-of-automation-in-enhancing-multi-property-management-systems.

[7] Deloitte, "2024 commercial real estate outlook: Finding terra firma," Deloitte Center for Financial Services, 2024.

[8] Sandhu, Ravi S.; Coyne, Edward J.; Feinstein, Hal L.; Youman, Charles E., "Role-Based Access Control Models," vol. 29, 1996.

[9] Voigt, Paul; Von dem Bussche, Axel, The EU General Data Protection Regulation (GDPR): A Practical Guide, Springer International Publishing, 2017.

[10] Blut, Markus; Wang, Cheng; Schoefer, Klaus, "Factors Influencing the Acceptance of Self-Service Technologies: A Meta-Analysis," *Journal of Service Research,* vol. 19, no. 4, 2016.

[11] IBM, "IBM," 2020. [Online]. Available: https://www.ibm.com/topics/enterprise-systems.

List of Figures

Figure 1 Team Roles Table 14
Figure 2 Gantt Chart Table 16
Figure 3 UML Diagram 18
Figure 4 ERD Diagram 19
Figure 5 Use Case Diagram for PAMS 20
Figure 6 Tenant's Dashboard 21

Figure 7 Administrator's Dashboard 22
Figure 8 Manager's Dashboard 22
Figure 9 Front Desk Staff's Dashboard 23
Figure 10 Maintenance Staff's Dashboard 23
Figure 11 Finance Manager's Dashboard 24
Figure 12 Reponse when incorrect input occurs 26

IX. Appendices

This section contains supplementary materials referenced throughout the report. These include:

- **Gantt Charts**

- Different versions of the Gantt chart showing project planning at various stages, including initial planning, mid-project updates, and final adjustments.
- These charts illustrate how tasks were allocated, timelines for frontend, backend, testing, and integration phases, and any revisions made during the project.

- **System Diagrams**

- UML diagrams, including Use Case diagrams and Class diagrams, used in the design phase to map system structure and user interactions.
- Database schema diagrams show tables, relationships, and normalization.

- **Screenshots of the Application**

- Key UI screens for different roles (Tenant, Administrator, Maintenance, Finance, Manager, Front Desk).
- Screens showing workflows such as login/authentication, CRUD operations, and dashboards.

- **Testing Artifacts**

- Examples of core test cases executed during the project, including authentication tests, role-based access control (RBAC) verification, and CRUD operations testing.
- Screenshots or tables of results demonstrating successful integration of frontend and mock data.

- Reference Resources and Learning Materials

- **Python Documentation:** W3Schools Python Tutorial – Used as a primary reference for syntax, data structures, and basic programming concepts during development.

  https://www.w3schools.com/python/default.asp

- **Flet Documentation:** Official Flet Developer Guide – Consulted for UI component usage, layout design, and interactive elements in Python.

  https://flet.dev/

- **Github:** Our project on Github:

  https://github.com/ductrung3004/Systems-Developemt-Group-Project

- Additional Materials

- Any supplementary charts, logs, or reference screenshots that help verify and illustrate the development process.